

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PY THON PROGRAMMING

ASSIGNMENT – 3

Name : **Hemnath R**

RegNo : **1920260116**

Course Code : **CSA0822**

GitHUB Link : https://github.com/hemnathr08-commits/python-programming-assignment_

Q1 – Lists and Slicing: Daily Rainfall Records

Problem: Avillagerain-gauge stationstores 365dailyrainfall readings inmillimetres. The panchayat needs weekly totals, wettest and driest readings with positions, 7-day moving averages and dry-day positions.

Design and implementation

• Store the daily values in a list. A weekly total is calculated from each seven-element slice readings[i:i+7].

• Use max() and min() to obtain extreme rainfall values; index() gives the first zero-based position.

• The 7-day moving average has len(readings)-6 results. For each starting position, average the corresponding seven readings.

• A dry day is any position whose reading equals 0 mm. enumerate() returns both the position and value.

Slicing interpretation and data-structure choice

readings[7:14] returns positions 7 through 13 (seven elements). readings[-7:] returns the final seven values. readings[::7] selects every seventh value; it is not a weekly total. A list is suitable because the year's readings are ordered and indexed. append() is amortized O(1), insert() is O(n) because elements may shift, pop() at the end is O(1) while pop(i) is O(n), and sort() is O(n log n) and changes the order.

Moving-average analysis

The naive method sums each seven-value window independently. For n readings and fixed window k=7, it performs about k(n-k+1) additions, O(nk), which is O(n) for fixed k. The running-sum method calculates the first window once, then adds the new reading and subtracts the reading leaving the window: S(i+1)=S(i)+readings[i+7]-readings[i]. It needs about 2(n-7) updates and is O(n), with O(n) output space.

Experimental investigation: The program generates 365, 36,500 and 365,000 readings with random. It times both implementations with time.perf_counter(), reports the results and checks empty and fewer-than-seven-element inputs. Timings depend on the computer and Python version; run the supplied program to obtain your machine's measurements.

Alternative: collections.deque(maxlen=7) is useful for a continuously arriving stream because it maintains a fixed-size window. A dictionary keyed by date helps date lookup but does not naturally support contiguous slicing. A running sum is O(n) time; storing all readings and averages uses O(n) space.

Late reading decision: list.insert(position, value) is O(n) due to shifting. For a year of only 365 readings, a list remains practical and simple. For frequent streaming updates, a deque may be preferable; for date lookup, pair the ordered list with a date-to-position index.

Pseudocode:

IF number of readings < 7: return empty result
sum7 = sum(first 7 readings); append sum7/7
FOR i from 7 to n-1: sum7 = sum7 + readings[i] - readings[i-7]; append sum7/7

Conclusion: Lists and slicing provide clear access to daily and weekly records. The running-sum moving average avoids repeated addition and scales better for large reading sets.

Q2 – Tuples, Mutability and Copying: City Bus-Stop Map

Problem: Abus service stores eachstop's GPScoordinatesas alatitude/longitudepairandusesa rows-by-columns grid to count passengers. A faulty grid repeats the same row object.

Implementation and observed aliasing bug

The expression bad = [[0] * cols] * rows creates one inner list and repeats references to it. Updating bad[0][1] therefore changes the corresponding position in every row. The corrected comprehension [[0 for _ in range(cols)] for _ in range(rows)] creates a new inner list on every iteration. The program prints both grids and uses is and id() to demonstrate object identity.

Tuples such as (13.0418, 80.2341) are immutable and hashable, so they can be dictionary keys. Lists are mutable and unhashable, so a list coordinate cannot be used as a dictionary key. Tuple unpacking in a loop assigns its two elements to latitude and longitude.

Copying

alias = original creates another reference to the same outer list. copy.copy(original) and original[:] create shallow copies: the outer list is new, but nested lists are shared. copy.deepcopy(original) recursively copies nested objects, so later inner-list changes do not affect it. A tuple cannot have an element reassigned, but it may contain a mutable list whose contents can still change.

A shallow copy is sufficient when the data contains immutable elements or when nested objects are intentionally shared. A deep copy is needed when independent nested mutable objects are required. Passing a list to a function passes an object reference; mutating the list in the function changes the caller's list. Rebinding the local parameter does not rebind the caller's variable.

Complexity and experiment

List indexing and tuple indexing are O(1) on average. List membership is O(n); set/dictionary membership is O(1) average. The program compares sys.getsizeof() for a list and tuple containing one million integers, and times construction/indexing. Exact byte counts and timings are platform-dependent. sys.getsizeof() reports the container's shallow size, not the total memory of referenced objects.

Alternative: A sparse dictionary keyed by (row, column) stores only occupied zones. Lookup and update are O(1) average, with O(z) space for z occupied zones, compared with O(rows×cols) space for a dense grid. A dense grid is simpler and efficient when most zones are occupied.

Conclusion: Tuples protect coordinate keys from accidental modification; independent row creation prevents aliasing. Choose shallow or deep copying according to whether nested mutable data must be independent.

Q3 – Dictionaries: Farmer Helpline Complaint Analysis

Problem: Analyse complaint text to find frequent words, complaints by district and crop, and the top k words after removing common words.

Implementation

- Convert text to lowercase, split into words and remove punctuation.
- Ignore stop words using a set. Count each remaining word in a dictionary using get() or setdefault().
- Group complaint records by district in a dictionary whose values are lists.
- Sort word-count pairs by negative count and then alphabetically: key=lambda item: (-item[1], item[0]).

For the supplied sample, the top three are [('seeds', 3), ('price', 2), ('received', 2)]. 'price' appears before

'received' because both have count 2 and alphabetical order breaks the tie.

Analysis and experiment

Dictionary lookup is O(1) average because a hash table computes a key's bucket. Counting n total words with d distinct words is O(n) average time and O(d) space. Repeated list.count() for every distinct word scans the full list each time, O(n×d), and can approach $O(n^2)$. collections.Counter is a specialised counting tool with comparable O(n) average behaviour. Python 3.7+ dictionaries preserve insertion order.

keys() provides a dynamic view of keys, values() a dynamic view of values, and items() a dynamic view of key-value pairs. The program generates 1,000, 10,000 and 100,000 complaint records and compares dictionary counting, list.count() and Counter. It also tests empty input and case/punctuation variants. Timings are generated when the code runs.

Deployment decision: For 100,000 complaints per day, use dictionary counting or collections.Counter, not repeated list.count(). Both scale linearly on average and use memory proportional to distinct words.

Conclusion: Dictionaries are suitable for frequency counting and district grouping because they provide fast average key lookup and direct key-value associations.

Q4 – Sets: Elective Registration Clash Detection

Problem: Find students registered in both electives, exactly one elective, or neither elective.

Set design

Represent each elective register as a set of student register numbers. Sets automatically remove duplicates. Use intersection (&) for both, symmetric difference (^) for exactly one, and difference (-) after union (|) for neither. Sort each result before printing so the report is reproducible.

For the screenshot's sample data, the expected results are: clash ['192311004', '192311009']; exactly one ['192311001', '192311002', '192311007', '192311010']; neither ['192311003', '192311005', '192311006', '192311008'].

Complexity and experiment

Set membership is O(1) average, while list membership is O(n). For sets of sizes a and b, intersection/union/difference are typically O(a+b) average; exact implementation costs depend on operand sizes. The program checks 10,000, 100,000 and 1,000,000 register numbers and compares membership in a list and a set. It also tests empty and identical electives.

A set discards duplicates and does not preserve a meaningful registration order. If the original order or duplicate count matters, retain a list as well. Sets cannot contain mutable lists, but can contain immutable tuples.

Conclusion: Sets are the appropriate structure for detecting clashes and producing unique sorted reports.

Q5 – List Comprehension: Cleaning Sensor Readings

Problem: Avaccinecold-storagemonitorrecords Fahrenheitreadings.Valuesabove 50°C orbelow -50°C are sensor errors. Convert valid values to Celsius and identify minute positions outside the safe 2–8°C range.

Implementation

The conversion formula is C=(F−32)×5/9. Filter valid sensor readings using -50 ≤ value ≤ 50 in the stated input scale, convert the retained values, and use enumerate() to produce the minute positions where Celsius temperature is below 2 or above 8.

The program implements three cleaning versions: a for loop with append(), a list comprehension with a condition, and map() with filter() and a lambda. It verifies that all three produce identical results. It also demonstrates a dictionary comprehension and a set comprehension.

Analysis and experiment

General conditional list-comprehension form: [expression for item in iterable if condition]. A comprehension is concise for simple transformations and filters; a loop is often easier to read when several statements, error handling or complex branching are required. A generator expression produces values lazily and typically uses O(1) additional iterator memory, while a list stores O(n) references.

The program times all three approaches for 10,000, 100,000 and 1,000,000 readings and compares sys.getsizeof() for a list and generator object. The reported generator size is the generator object's shallow size; it does not include the source list. All-valid and all-invalid test cases verify boundary behaviour.

Conclusion: List comprehension is readable and effective for simple filtering, but it is not always best. A generator is preferable when processing a large stream without needing all results stored at once.

Q6 – Nested Data: A Railway Timetable

Problem: Storea timetableas a nesteddictionarymappingtrainnumberto train name, a list of (station, arrival, departure) tuples, and a set of running days.

Implementation and nested access

The outer dictionary is keyed by train number. Each train's inner dictionary stores name, stops and running_days. Each stop is a tuple. The expression timetable[train_number]['stops'][2][2] retrieves the departure time from the third station (zero-based index 2).

The program builds at least five trains, searches for trains stopping at a given station on a selected day, sorts matches by departure time using sorted(..., key=lambda item: item[0]), and prints the timetable neatly. A station index maps a lowercased station name to a list of train numbers.

Query cost and index trade-off
Without an index, one station query scans trains and their stops: O(total stops) time. With an index, finding candidate trains is O(1) average dictionary lookup plus O(number of trains at that station) to retrieve details; the index costs O(total stops) additional space to build and maintain. Indexes speed repeated queries but require extra memory and updates when timetable data changes.

The program generates 100, 1,000 and 10,000 trains and times repeated station queries with scanning and with index construction plus indexed lookup. It tests an unknown station and a day on which no matching train runs.

Alternative: A flat list of (train, station, arrival, departure, days) records is easy to export and scan, but every query may scan all records. Nested dictionaries are convenient for direct train lookup; a station-to-train index is useful when station queries are frequent.

Conclusion: Nested structures represent the timetable naturally. An additional station index is worthwhile when query frequency justifies its memory and maintenance cost.

Q7 – Design a Contact-Merging Program from First Principles

Problem: Merge contacts from two old phones and a SIM card. The same person may appear with different name spellings and phone-number formats. Each person should appear once with all associated numbers and email addresses.

Stage 1 – Requirements

• Input: multiple contact lists containing optional name, phone numbers and email addresses.

• Output: merged contacts, each with collected names, normalized phone numbers and normalized email addresses.

• Normalize names by trimming and case-folding whitespace; normalize phone numbers by retaining digits and removing a leading country code 91 from a 12-digit Indian number; normalize email by trimming and case-folding.

• Use normalized phone number as the primary dictionary key. Retain records without a phone number using normalized name and email as a fallback key.

Stage 2 – Design pseudocode

CREATE empty dictionary merged and fallback dictionary
FOR each source list and each contact:
normalize name, every phone and every email
IF valid phone numbers exist: find an existing group by any normalized number or create one; add names, numbers and emails to sets; map every number to that group
ELSE: group by normalized name and normalized emails
REMOVE duplicate references to the same group and return unique records.

Stage 3 – Implementation and measurement

The complete program normalizes input, merges records with dictionaries and sets, verifies equivalent phone/email formats, and measures merging 1,000 and 100,000 generated contacts. Dictionary grouping is O(n) average for n contacts (assuming average O(1) dictionary operations), with O(n) storage for the merged records and identifiers.

Stage 4 – Defend the design

A dictionary keyed by normalized phone number avoids comparing every pair of contacts. Pairwise comparison is $O(n^2)$, while hash-based grouping is O(n) average. Sets prevent duplicate phone numbers and email addresses. A limitation remains: two different people may share a phone number, and the same person may have no shared phone number across devices. Name-only matching can incorrectly merge different people. A production system should use stronger identity confirmation and handle country-specific phone formats.

Alternative analysis: Pairwise duplicate detection compares each contact with every other contact and costs $O(n^2)$ time; dictionary grouping is O(n) average time and O(n) space. Actual performance depends on normalization, collisions, input quality and hardware.

Conclusion:

Normalize identifiers before merging, use a dictionary for efficient grouping, and use sets to accumulate unique values. The matching rule must be treated as a practical heuristic rather than proof that two records belong to the same person.